PRISE EN MAIN DU DATASET FIL ROUGE

Voici une série d'exercices sur la prise en main et l'utilisation d'un dataset :

Exercice 1

- Aller dans BigQuery, créer un dataset que vous nommerez 'dataset_fil_rouge' et upload depuis votre ordinateur les 7 fichiers du zip "dataset_fil_rouge" du cours.
- En tant qu'Analytics Engineer, il est important de savoir rapidement identifier les relations entre les différentes entités/objets de la base de données.
  Faire un diagramme entité relation simple (sans mettre les noms de colonnes) de ce dataset : représenter uniquement les noms des tables, les clés primaires/étrangères ainsi que les cardinalités des relations entre ces tables.

Exercice 2

Votre collègue qui vient d'arriver dans l'entreprise a écrit ses premières requêtes SQL et il vous demande votre avis dessus.
Reprendre les 4 requêtes et proposer une version améliorée pour chacune (que ce soit dans le fond ou dans la forme).

DataBird 2024 ©copyright

Il faudra être attentif à :

- Noms des colonnes
- Indentation
- Optimisation

Query 1

```
SELECT
COUNT(*),
COUNT(DISTINCT(user_name))
FROM (
SELECT
  order_id,
  user_name
FROM
  sales_database.order
WHERE
  order_status = 'created');
```

Query 2

```
SELECT product.product_id,
seller.seller_zip_code,
order_item.price,
order_item.shipping_cost,
orders.order_id
FROM sales_database.product
LEFT JOIN sales_database.order_item ON order_item.product_id =
product.product_id
LEFT JOIN sales_database.seller ON seller.seller_id =
order_item.seller_id
LEFT JOIN sales_database.order AS orders ON order_item.order_id =
orders.order_id;
```

DataBird 2024 ©copyright

Query 3

```sql
WITH monthly_users_recap AS (

SELECT DATE_TRUNC(order_date,month) AS order_month,
COUNT(DISTINCT user_name) AS total_monthly_users
FROM sales_database.order
GROUP BY order_month
ORDER BY total_monthly_users DESC

), total_monthly_user_from_jawa_timur AS (
SELECT DATE_TRUNC(order_date,month) AS order_month,
COUNT(DISTINCT user.user_name) AS total_monthly_users_from_jawa_timur
FROM sales_database.order AS orders
LEFT JOIN sales_database.user AS user ON user.user_name =
orders.user_name
WHERE user.customer_state LIKE '%JAWA%TIMUR%'
GROUP BY order_month

), monthly_orders_recap AS (

SELECT DATE_TRUNC(order_date,month) AS order_month,
COUNT(order_id) AS total_monthly_orders
FROM sales_database.order
GROUP BY order_month

), shipping_cost AS (

SELECT shipping_cost
FROM sales_database.order_item
WHERE price > 7000
)
SELECT u.order_month,
COALESCE(u.total_monthly_users,0),
COALESCE(jt.total_monthly_users_from_jawa_timur,0),
COALESCE(o.total_monthly_orders,0)
FROM monthly_users_recap AS u
LEFT JOIN total_monthly_user_from_jawa_timur AS jt ON jt.order_month =
u.order_month
LEFT JOIN monthly_orders_recap AS o ON o.order_month = u.order_month
ORDER BY order_month
```

DataBird 2024 ©copyright

Query 4

```
WITH orders_filtered AS (
 SELECT *
 FROM sales_database.order
 WHERE order_status = 'shipped'

)

SELECT COUNT(DISTINCT product_id) AS total
FROM orders_filtered
INNER join sales_database.order_item ON order_item.order_id =
orders_filtered.order_id
```

Exercice 3

Écrire les queries SQL les plus optimisées possibles pour répondre à chacune des questions. Il faut mettre des noms de colonnes explicites et en anglais pour chaque métrique calculée.

Certaines order ne sont pas valides et ne doivent pas être prises en compte dans ces calculs.
Utilisation de ROW_NUMBER(), COALESCE, ROUND pour que les résultats soient propres.

Dans cet exercice, considérez que les tables sont propres et dédupliquées, ne procédez pas à des opérations de dé-duplication des lignes.

1) Faire une query SQL qui donne pour chaque **order Dimension**

- order_id
- order_status
- order_created_at
- order_approved_at

DataBird 2024 ©copyright

Métrique

- La valeur totale de la commande
- Le nombre total d'item
- Le nombre total d'item distinct
- Le feedback score moyen

2) Faire une query SQL qui donne pour chaque **utilisateur**
   - La valeur totale des commandes faites pour l'utilisateur
   - Le nombre de commandes de cet utilisateur
   - Son produit favori (produit favori = le produit qui a été acheté le plus de fois par le client)

Pour chaque utilisateur, nous voulons récupérer le user_name et la customer_city.

3) Faire une query SQL qui donne pour chaque **produit**
   - Le nombre distinct de clients ayant acheté ce produit
   - La région la plus dominante dans les clients ayant acheté ce produit
   - Le volume du produit

Pour chaque produit, nous voulons récupérer le product_id, le product_category,

4) Faire une query SQL qui donne pour chaque **vendeur**
   - Total des ventes (price + shipping_cost)
   - Le score moyen de feedback des commandes dans lesquelles le vendeur est impliqué

Exercice 4

1) J'ai plusieurs tableaux de bord clés connectés à ma table partitionnée dans Metabase, mais mes coûts n'ont pas diminué comme prévu. Quelles actions pourrais-je entreprendre ?

DataBird 2024 ©copyright

2) Votre entreprise utilise Metabase pour les dashboards, connectés à des tables BigQuery. Cependant, vous avez remarqué que de nombreux Data Analysts exécutent des requêtes non optimisées directement dans BigQuery, ce qui augmente les coûts et impacte les performances.

En tant qu'Analytics Engineer, vous devez suivre quelles requêtes sont exécutées (via Metabase ou directement dans BigQuery), analyser l'impact des requêtes en termes de coût afin de suivre et contrôler le budget de l'entreprise.

Quelle stratégie pourriez-vous adopter pour accomplir ces tâches ?

Exercice 5

Partitionnement sur des petites tables

Lien du dataset à télécharger :
https://www.kaggle.com/datasets/valentinjoseph/avocado-sales-20152021-us-centric

1) Comment est structurée cette table ?
2) Que représente une ligne dans cette table ?
3) Écrire une requête SQL qui calcule, par année, pour les avocats de type "Organic" dans les régions suivantes : 'Chicago', 'Charlotte', 'Sacramento', 'SanDiego' ,'SanFrancisco'
   - Le nombre total d'avocats vendus
   - Le prix moyen des avocats vendus (en calculant une moyenne pondérée avec le volume)
4) La requête que vous venez de rédiger est connectée au dashboard principal de l'entreprise dans Metabase, et ce dashboard coûte très cher. Votre manager vous a donc demandé d'explorer les méthodes de partitionnement et de clustering pour réduire potentiellement ces coûts.

   Explorez les deux méthodes (partitionnement et clustering) sur la table des ventes d'avocats et choisissez les partitions et clusters qui auront un impact direct sur les coûts de votre requête.

DataBird 2024 ©copyright

Exercice 6

Audit des Tables de Grande Taille et Analyse des Requêtes BigQuery

INFORMATION_SCHEMA dans BigQuery fournit des informations sur les métadonnées de votre projet. Ces informations sont essentielles pour le suivi des coûts de stockage, la gestion des ressources et l'audit des performances de vos requêtes. En accédant aux vues de INFORMATION_SCHEMA, vous pouvez obtenir des détails sur les tables, les partitions, ainsi que les statistiques d'utilisation des requêtes.

Lien vers la documentation :
https://cloud.google.com/bigquery/docs/information-schema-intro?hl=fr

Question 1 : Audit des Tables de Grande Taille

Écrivez une requête qui permet d'obtenir la liste de toutes les tables présentes dans votre projet BigQuery, dans la région US. Pour chaque table, lister les informations suivantes :

- Le nombre de lignes de la table.
- Le nombre total de mégaoctets logiques (logical megabytes).
- Le timestamp de création de la table.
- Le timestamp de la dernière modification de la table.
- Un booléen indiquant si la table est partitionnée ou non.
- Filtrer les résultats pour ne récupérer que les tables créées dans les deux derniers mois.

Question 2 : Analyse des Requêtes Utilisateur

Écrivez une requête qui, toujours pour la région US, donne pour chaque jour et chaque email d'utilisateur, uniquement pour les requêtes de type SELECT :

- Le nombre total de bytes facturés (bytes_billed).
- Le nombre total de requêtes effectuées.
- Le nombre total de requêtes ayant généré une erreur.

DataBird 2024 ©copyright

Exercice 7

1) Quelle est la différence entre les fonctions RANK() et ROW_NUMBER() ?

2) Analyse des transactions Bitcoin
La table bitcoin_raw_datas contient les informations des transactions Bitcoin. Les données sont enregistrées chaque seconde.

Utilise les fonctions RANK() ou ROW_NUMBER() dans une ou plusieurs requêtes optimisées pour identifier, chaque mois, les heures où le volume de transactions est le plus élevé.

3) La volatilité peut être définie comme l'écart entre le prix le plus élevé et le prix le plus bas sur une période donnée.Est-ce que l'heure ayant la plus forte volatilité est aussi celle où le volume de transactions est en moyenne le plus élevé ?

Écris une requête pour commencer à répondre à cette question et compare la volatilité et le volume des transactions par heure.

DataBird 2024 ©copyright